The build loop as a lab method: an agentic worklist with independent tests, and a protein-model experiment run under it

Mang-yin Mo*
Independent researcher
mangyinm@alumni.cmu.edu

Draft, October 30, 2026

Abstract

An agent that writes code and then grades its own work will, sooner or later, report a pass for a property that is absent. We describe a build loop in which that cannot happen by construction: a worklist tool owns every state change, each task's test is written by an author who never sees the implementation, the implementer never sees the test, and a task closes only when the tool itself observes the test pass. We teach the loop as a lab method — a specification as pre-registration, the independent test as blinded assessment, run records and traces as the notebook, lessons as protocol amendments adopted only between passes — because that is how a computational biologist already reasons about trust. As the worked experiment we run a protein-language-model evaluation inside the loop: an aviary environment exposes an ESM-2 scorer to an agent, which chose 66 measurements over 5 rounds and named 7 positions as tolerant controls. Every number in this paper is bound to an evidence file by a check that fails the build otherwise; each carries a label saying whether it was re-measured for this paper, reconstructed from version control, reported from the original run, or documented. We close with a taxonomy of 22 failure modes found by auditing our own trust core, each with the regression test that now pins it and a laboratory analogue. Self-improvement is a mechanism of the loop, not a result of this paper: pass-to-pass improvement was never measured, and we say what measuring it would take.

1 Introduction

Computational biology has learned, expensively, that a pipeline's most dangerous output is a number that looks measured and was not: a held-out set the model had seen, a QC step that was skipped and logged as passed, a model whose identity was never recorded [3]. Agentic coding tools now produce pipelines, and they produce this failure at a new scale, because an agent can also write the test, run it, and report the result — three roles that a laboratory keeps apart on purpose.

This paper makes one argument and shows one experiment. The argument is that an agent's build should be run the way a careful lab runs an experiment, and that the discipline transfers almost term for term (Section 2). The experiment is a small protein-language-model study (Section 4), chosen because its answer is known in advance and so any failure of the loop shows up as a wrong number rather than a plausible one. Section 3 describes the tool that implements the loop, Section 5 the failure modes we found by auditing it, and Section 6 what this paper does not establish.

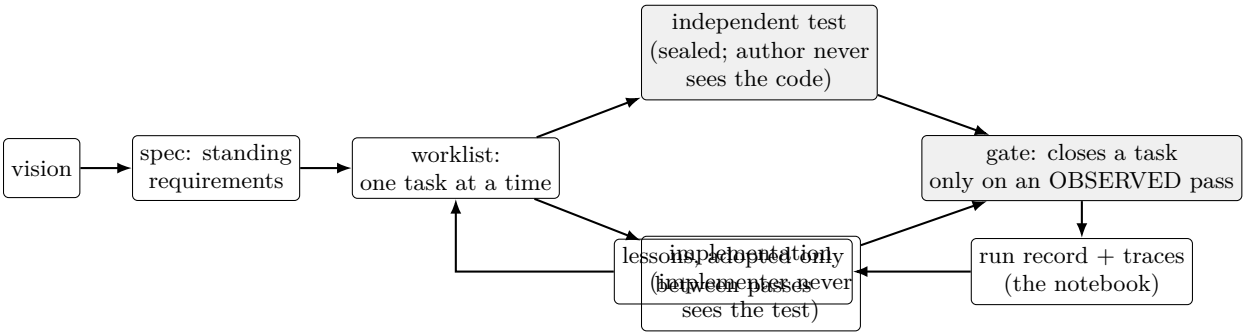
Two commitments shape everything that follows. First, every numeral in the text is a macro bound to a file under `paper/evidence/`, and a script refuses to build the paper if a number is unmapped, a file is missing, a value disagrees, or a claim that was only *reported* is phrased as *measured*. That check reads numerals: a count the text spells out in words is not bound by it, which we state here rather than imply a coverage the check does not have. Second, each number carries one of six labels — *measured* (re-run for this paper, on

*ORCID 0009-0009-5233-3142.

a stated date, with its configuration recorded), *reconstructed* (rebuilt from version control), *reported* (taken from the original run's artifact), *documented* (a taxonomy row citing its source), *queried* (an external store answered on a stated date), *recovered record* (a copy found after the fact, checked for consistency, its time of writing not established) — and the appendix lists them all, with the script refusing a label the evidence file does not itself carry.

2 The loop as a lab method

The loop takes a free-text vision to a reviewed branch of working, individually tested code. Its stages are ordinary; what is unusual is where the trust boundary sits.



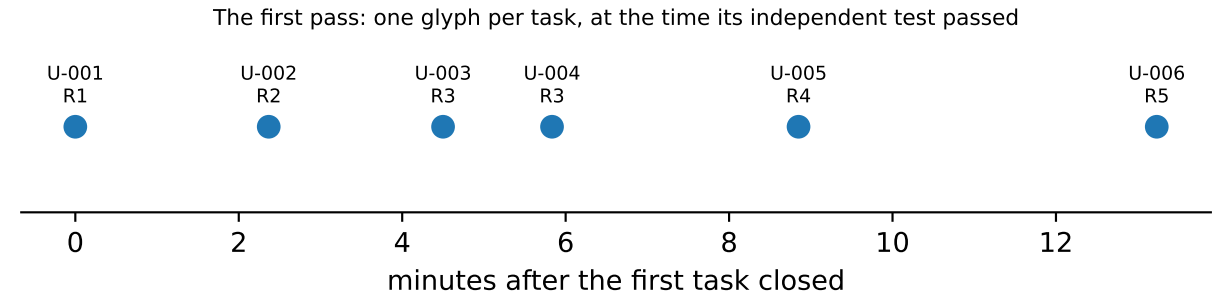
Lab method	The loop
Pre-registration	The specification: standing requirements, frozen at one approval before any code is written. Scope cannot drift mid-pass.
Blinded assessment	The independent test for each task, written by an author who never sees the implementation. The implementer never sees the test.
The lab notebook	The pass record and the traces: what was attempted, what closed, what was set aside, under which configuration.
A protocol amendment	A lesson: a captured observation that may change how the next pass runs, and is adopted only between passes, never during one.
The reviewer who signs off	The gate: a worklist tool with no model inside it, which closes a task only when it has itself observed the independent test pass.

Table 1: The mapping this paper teaches by. The right-hand column uses the plain words the text uses throughout; the project's own glossary maps each to its internal term.

Who can close a task. Nothing that writes code can record that it passed. The worklist tool is the only writer of a task's state; it runs the independent test itself and reads the test runner's own report, never the runner's exit code, because exit codes conflate "the test failed" with "the test never ran". A gate that could not execute halts and says so; it is never retried as if the code were merely wrong, because that path burns the attempt budget on nothing and then sets a task aside with evidence of nothing. This distinction — *ran and failed* versus *could not run* — recurs in Section 5 as the shape of most of the defects we found.

The seal. The independent test is sealed: a digest is recorded when it is authored, the gate refuses a test that changed after sealing, and the agent that implements is blocked by a hook from reading the sealed directory at all. When this paper needed the sealed suite re-run (Section 4), the run happened through the same referee tool the gate uses, over an archived copy of the pass-end commit, and the authors saw counts and test names only.

What the first pass looked like. The first pass closed 6 tasks against a small spend-tracking library, in the order their requirements were declared, with each task's independent test passing on the tool's own observation (Figure 1). The timeline is *reconstructed* from version control: each close is a commit whose subject names the task and the requirement it satisfies. Section 6 explains why the tool's own record of that pass carries a weaker label.



generated from paper/evidence/drain1-from-git.csv at d7ea517

Figure 1: The first pass, reconstructed from version control: one glyph per task at the time its independent test passed. 6 tasks. Whether any task was set aside is not something version control records; the recovered record says none was (Section 6).

3 The tool

The loop ships as a plugin for an agentic coding harness, with three command-line tools that contain no language model:

- **the worklist** (`register`) owns every task transition — claim, seal, close, set aside — and is the only thing that may close a task. It records the pre-claim commit so a task set aside can be restored exactly, and writes the pass record at the end of a pass.
- **the referee** (`referee`) runs a project's sealed suite through a configured command, reads results in a fixed dialect, counts what arrived against what the suite declared, and signs a receipt only for a suite that passed with nothing skipped and nothing unaccounted for.
- **the meter** (`spend`) enforces a declared spending cap; it was the loop's own first product, built by the first pass.

At the version this paper cites, the plugin's own suites for these tools run 219 assertions with 0 failing (*measured*: re-run for this paper in a scratch checkout of the tagged commit, not copied from a changelog); the worklist tool alone carries 60 of them. That version is `aiadlc` 0.64.0 (commit `e54bb44`); the plugin was renamed `airdlc` after it, so later releases are published under that name. Installation is a plugin-marketplace add followed by one configuration key naming the project's referee command; the plugin's repository documents the first, and the evidence README gives the key and the command this paper used.

The tool is deliberately small. Its safety does not come from the agents being trustworthy; it comes from the fact that no agent in the loop can close its own work, and from the referee reading a report rather than an exit code. Section 5 is a list of the ways that small tool was nevertheless wrong.

4 The worked experiment

The question. Human preproinsulin folds through three disulfide bonds. Mutations that disrupt or create a cysteine in proinsulin, C96Y among them, cause permanent neonatal diabetes through misfolding [7], and

the Akita mouse's Ins2 C96Y allele is the classic disulfide-disrupting case [9]. So a protein language model that has learned anything about this protein should treat those six positions as load-bearing. This is a question with a known answer, which is exactly what one wants from a worked example: if the loop had corrupted the number, the number would be wrong, not merely different.

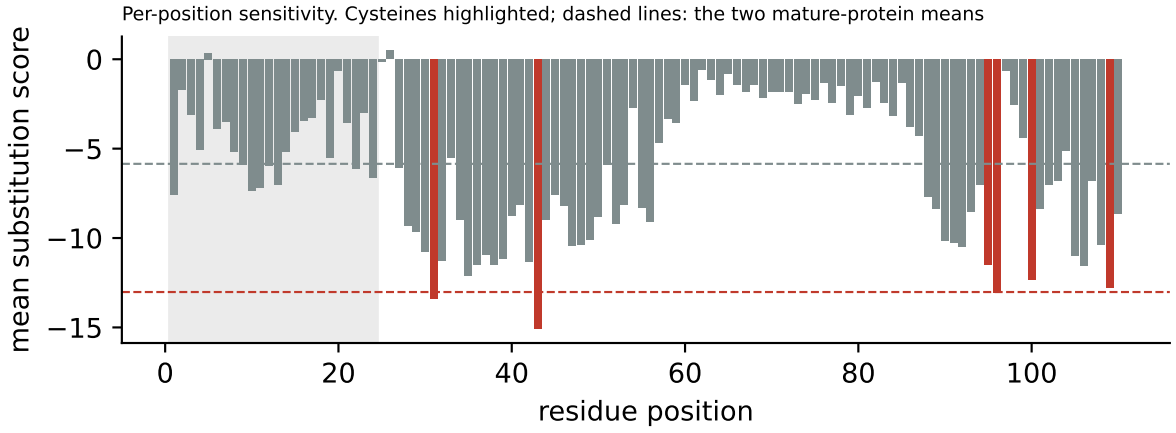
The environment. We wrap an ESM-2 650M scorer [4] as an aviary environment [6]: the agent sees two tools, one that scores a single-residue substitution and one that embeds a sequence, and nothing else. Scores are masked-marginal log-odds [5]; the evaluator is frozen and its identity recorded. The sequence is fetched from UniProt [8] by accession and cached; the cache is file content, not trusted input, and every value read from it is validated before it can reach a path or a URL.

The input, verified. Before any number was re-computed we fetched the record fresh from the database with the cache bypassed and compared it, by digest, to the sequence the original run used: 110 residues, identical digest (*measured*, fresh fetch, dated in the evidence file). The input is UniProt accession `P01308` at database release `2026_03`, with sequence SHA-256

`99603f8fb1e9872e2d24ee1e9f85108c2bddea6b148a0f1099e16ba2715c8263`.

Within this paper these three values appear only here and in the evidence file, by an explicit ruling: a digest over an enumerable identifier is a lookup, not a redaction, and the association between this protein and its public record is already published by the repository, so restating it for reproducibility creates no new one.

Results, re-measured. Over the hash-verified input we re-ran the full single-substitution scan — 2,090 substitutions — with the model identity, weights digest, device, seed and library versions recorded beside the numbers. Every headline value matched the original run to the reported precision, and all per-position means matched exactly. Within the mature protein (1,634 substitutions after the signal peptide), the six disulfide cysteines score −13.02 on average against −5.85 for every other residue (Figure 2); across the whole precursor, signal peptide included, the non-cysteine mean is −5.53 instead, and the paper is careful throughout about which of the two a given number is. The re-run took 12.70 seconds on an Apple GPU. The original run's own artifact reports 7.30 seconds (*reported*). The model weights were 2,609,506,392 bytes, digest recorded.



generated from paper/evidence/science.json at d7ea517

Figure 2: Per-position mean substitution score across the 110-residue sequence; cysteines highlighted; the signal peptide shaded. Dashed lines: the mature-protein means, −13.02 at the cysteines and −5.85 elsewhere (*measured*, re-run).

The agent's run. A discovery agent given the environment and a one-paragraph objective chose 66 measurements over 5 rounds, at 34 distinct positions, spending 15,115 completion tokens in the rounds that recorded a count (*reported*: counted from the run's own artifact, whose digest and size are in the evidence; 1 round recorded no token count and is reported as such, not as zero). Round 1, its first, requested every one of the six cysteines, and two C-peptide positions with them, so the cysteines were targeted from the start rather than arrived at: the objective named the protein, and where to look was prior knowledge the agent brought, not a finding it made. In its final write-up it listed 7 positions as tolerant controls, of which 5 fall in the C-peptide — cleaved out of mature insulin and poorly conserved — as the scan's own region annotation defines it, the rest lying outside it. The agent never refers to that region; reading its choice as a C-peptide control is our interpretation of what it did, and the only thing *reported* here is the list of positions. At the C-peptide positions among them, in sequence order, the re-run scan scores the alanine substitution at 1.53, 0.18, 0.17, 1.08 and −0.10, against −13.02 at the cysteines (*measured*). Which positions to probe was the agent's choice, and is *reported*; which substitution it tried at each is not in the evidence, so none is claimed. The run's trace exists in the project's trace store: 1 root trace for the discovery run, one of the project's 4 root traces, among 36 traces in all (*queried*, dated).

The independent tests, re-run. The 6 tasks of the first pass carried 75 independent tests. For this paper they were re-run through the referee over an archived copy of the pass-end commit, with every dependency declared: 75 passed, 0 failed, none skipped (*measured*, re-run, dated). The referee also signed a receipt, and that signature is deliberately not part of the claim: it binds the tree the referee ran in, not the archived pass-end tree the suite was run over, so it attests the counts' provenance and not the code they were measured against. Its hash is recorded in the evidence file. The authors saw counts and test names only.

5 What the audit found

We audited the loop's own trust core — the worklist, the referee, the isolation guard around the science suite, and the seeder that feeds the demonstration dashboard — across several review passes, and kept a table of every defect. They share one shape: *a mechanism reports success while the property it exists to guarantee is absent.* Table 2 (rendered in full as Figure 3) lists 22 of them; every row cites the regression test that now pins it or says plainly that it has none (*documented*: the check that builds this paper verifies each cited test exists). Those tests live in the repository's own suites, re-run for this paper: 328 over the science environment, 42 over the worklist tool, and 173 over the scripts that produce this paper's evidence (*measured*, re-run, dated).

The shape	The laboratory analogue
A gate read a runner's exit code, so "could not run" looked like "ran and failed".	A QC step marked FAIL when the instrument never switched on.
A test suite was green because a dependency happened to arrive through another package.	An undeclared reagent that came in another kit's box.
A guard's own tests could not fail; the mutation gate that checks the guard credited any non-zero exit as a kill.	A held-out metric that stays good after the model is ablated.
A demonstration seeder wrote a fixture into the live run directory, over the real record.	Simulated rows placed in the results directory.
A referee read a skipped test as a pass, and signed a receipt for a run that merely happened.	A certificate of analysis certifying only that an analysis was run.
A digest of an enumerable identifier was taken for a redaction.	Pseudonymisation that a lookup table reverses.

Table 2: Six of the 22 shapes, chosen for their analogues. The full table is Figure 3.

Two of the rows are about building this paper. The referee command we wrote to re-run the sealed suite first wrapped the test runner in a construct that swallowed its exit status, so "no tests collected" would have printed nothing and read as a pass — caught at the plan gate, before it ran. And the script that records the referee's verdict read the wrong output line under one flag, and recorded a signed, passing run as *not measured*: a false negative in the safe direction, still false, and it would have hidden a real pass. We include them because a taxonomy of other people's mistakes is easy to write.

The mutation gate deserves a sentence, because it is how several of these were found [2]. The isolation guard around the science suite is protected by a frozen catalogue of mutants — small edits to the guard that a correct test suite must notice. A survivor means a property is no longer pinned by any test. Three review passes each found the same defect in different clothes (authentication of a module by a marker, then by its file path, then by its declared origin — every attribute a fake can set), and each time the catalogue was extended rather than the count re-declared. The count is reported as live entries plus retired entries, never as a bare ratio, because a retargeted entry makes the count say something the catalogue cannot support: 48 of 48 live mutants killed, with 1 retired against a stated reason.

6 Limitations

Self-improvement is a mechanism, not a result. The loop captures lessons between passes and pins the lesson set a pass ran under. This paper ran one pass. Pass-to-pass improvement was never measured, lessons have not been shown to change an outcome, and no sentence in this paper claims otherwise. Section 7 says what measuring it would take.

The tool's own record of the first pass. The demonstration seeder once wrote a made-up fixture — more tasks, three passes, apparent improvement — into the same live directory the loop writes to, in the main checkout, over the first pass's own record. The fixture depicted exactly the claim the previous paragraph forbids; the public dashboard page disclosed it; the seeder now refuses that directory (Section 5). A copy of a record for the first pass was later recovered from a separate worktree. Its pre-claim chain checks out against history for all 6 tasks (each recorded pre-claim commit is exactly the parent of that task's close), it lists 6 closed and 0 set aside, and its recorded register digest matches the register file beside it. None of that establishes *when* it was written: a later reconstruction writing both files would also match, the tool's own provenance check predates the record's format and reports it unverified, and the tool is itself the artifact under study. We therefore label it *recovered record; pre-claim chain verified; write-time provenance not established*, and the pass itself as *reconstructed from version control*.

The check found a false provenance record in this paper's own evidence. A recorded commit identifies the producing code only if the working tree was that commit, and for one file it demonstrably was not. When the build was made to *enforce* that condition rather than merely record it, it refused the plugin-suite file: the commit recorded beside those numbers did not contain the script that writes them, because the script was first committed later, in the same commit that added the evidence. That run happened against an uncommitted producer and recorded whichever commit was checked out. The counts were right — re-running the suites at the same tag reproduces all of them exactly — but the provenance was not, and a reader who trusted the commit would have been reading the wrong tree. The file has since been rebuilt from the tag it names, on a clean tree, and now carries the same guarantee as the rest. We report the episode because it is the paper's argument turned on the paper: the defect was invisible for as long as the field was written and never checked, and it surfaced the moment something refused to build without it.

Both the model and the agent already knew the answer. The experiment was chosen because its answer is known, and that cuts the other way too: it establishes nothing about either system discovering anything. ESM-2's training set includes insulin, so the scan recovers a constraint the model has seen rather than predicting an unseen one. The agent was told which protein to study and requested all six cysteines in its first round, before any measurement came back (Section 4), so its targeting is prior knowledge carried into the task, not an inference drawn from the scores. What the experiment tests is the loop: whether a

known-correct number survives the machinery intact. A claim about discovery would need a protein held out of training and an objective that does not name the site of interest, and this paper makes none.

Compute and venue. The scan ran on an Apple GPU. Every agent call went to W&B Inference, the serverless OpenAI-compatible endpoint operated by Weights & Biases, a CoreWeave company, on `deepseek-ai/DeepSeek-V4-Pro-0813`; the two code paths that configure it are `science/run_discovery.py` and `science/pipeline.py`. The venue that framed the work supplied no compute for it. That sentence is about what was *granted*, and it needs the qualifier, because the endpoint's operator and the venue's host are the same company: the host documents this serverless inference as delivered through W&B Inference and running on its own GPU infrastructure [1]. So the calls were very probably executed on the host's machines. What the venue supplied was no hardware and no allocation made for this work. How the endpoint's usage was paid for is a separate question, and we do not settle it: the operator's documentation requires service credits for access, and whether those were bought, came with a plan's free tier, or were granted is not recorded in this repository. That gap is load-bearing rather than incidental: credits that were granted rather than bought would mean the operator supplied the compute after all, and it is then the no-compute sentence above that would have to change, not this qualification of it. We spell all of this out because the shorter sentence, standing next to a named provider, reads as the stronger claim we cannot evidence.

Budget enforcement never ran end to end. The environment refuses to start a discovery run without a declared token price, and none has been supplied; the agent run reported above predates that enforcement.

Reproducibility of the original science numbers. When the original numbers were computed, the sequence cache's read path had no integrity check. The re-run in Section 4 used an input verified fresh against the database and matched the original on every value, which is why those values now carry a *measured* label; the original artifacts alone would not have earned it.

A single pass, a small library. 6 tasks against a spend tracker is a demonstration of the mechanism, not evidence about how the loop behaves on a large codebase or over many passes.

7 Preconditions for measuring improvement

No second pass was run for this paper, and nothing here executes one. Measuring improvement would need: a second pass on the same specification with the lesson set pinned by content (the current pin is a tree digest over a store that background hooks rewrite, so two passes with identical lessons can carry different pins); the two prompts — the test author's and the implementer's — pinned alongside the seed, because the first pass's hardest task closed first try, and its prompt named the answer (T08) — which of those explains the other was never tested; a pre-declared outcome measure (tasks closed per attempt, or attempts to close) written down before the pass starts; and a run record whose authorship the tool's own provenance check can verify, which the tool now writes.

A Claims table

Every macro in this paper, its value, its label and the evidence field it resolves to. Generated by the claims check at build time.

id	value	label	evidence (file:field)
`subs`	2,090	measured (re-run, 2026-10-11)	`science.json:headline.substitutions`
`cysmean`	−13.02	measured (re-run, 2026-10-11)	`science.json:headline.cysteine_mean_mature`
`othermean`	−5.85	measured (re-run, 2026-10-11)	`science.json:headline.other_mean_mature`

id	value	label	evidence (file:field)
othermeanall	−5.53	measured (re-run, 2026-10-11)	science.json:headline.other_mean
matsubs	1,634	measured (re-run, 2026-10-11)	science.json:headline.n_substitutions_mature
seqlen	110	measured (fresh fetch, 2026-10-11)	science-input.json:published.length
ctrlA	1.53	measured (re-run, 2026-10-11)	science.json:headline.c_peptide_controls.62
ctrlB	1.08	measured (re-run, 2026-10-11)	science.json:headline.c_peptide_controls.79
ctrlC	−0.10	measured (re-run, 2026-10-11)	science.json:headline.c_peptide_controls.83
ctrlD	0.18	measured (re-run, 2026-10-11)	science.json:headline.c_peptide_controls.70
ctrlE	0.17	measured (re-run, 2026-10-11)	science.json:headline.c_peptide_controls.72
rerunsec	12.70	measured (re-run, 2026-10-11)	science.json:runtime_s
pubsec	7.30	reported	science.json:published_runtime_s
weightsgb	2,609,506,392	measured (re-run, 2026-10-11)	science.json:weights_bytes
meas	66	reported (artifact counted, 2026-10-11)	agent-run.json:measurements
rounds	5	reported (artifact counted, 2026-10-11)	agent-run.json:rounds
tokens	15,115	reported (artifact counted, 2026-10-11)	agent-run.json:completion_tokens_total
notokens	1	reported (artifact counted, 2026-10-11)	agent-run.json:rounds_without_a_token_count
ctrlnamed	7	reported (artifact counted, 2026-10-11)	agent-run.json:control_positions.named_by_the_agent_count
ctrlnamedcpep	5	reported (artifact counted, 2026-10-11)	agent-run.json:control_positions.named_by_the_agent_in_c_peptide_count
cysallround	1	reported (artifact counted, 2026-10-11)	agent-run.json:all_cysteines_requested_in_round
positions	34	reported (artifact counted, 2026-10-11)	agent-run.json:distinct_positions
traceroots	1	queried (2026-09-26)	weave-count.json:run_discovery_root_traces
tracetotal	36	queried (2026-09-26)	weave-count.json:project_total
evcommit	26da24de11f4470b7f2ddfb72e72cbae4a382d3d	measured (re-run, 2026-10-11)	science.json:provenance.git_sha
traceprojroots	4	queried (2026-09-26)	weave-count.json:project_roots
sealed	75	measured (re-run, 2026-10-11)	drain1-rerun.json:referee.total
sealedfail	0	measured (re-run, 2026-10-11)	drain1-rerun.json:referee.failed

id	value	label	evidence (file:field)
units	6	reconstructed	drain1-from-git.csv:rows
chain	6	recovered record; pre-claim chain verified	drain1-record-check.json:pre_claim_matches
closed	6	recovered record	drain1-record-check.json:closed_count
parked	0	recovered record	drain1-record-check.json:parked_count
pluginver	0.64.0	measured (re-run, 2026-10-11)	plugin-tests.json:plugin_version
plugincommit	e54bb44	measured (re-run, 2026-10-11)	plugin-tests.json:commit
plugintests	219	measured (re-run, 2026-10-11)	plugin-tests.json:totals.run
pluginfail	0	measured (re-run, 2026-10-11)	plugin-tests.json:totals.failed
regtests	60	measured (re-run, 2026-10-11)	plugin-tests.json:suites.register.test.sh.run
taxrows	22	documented	taxonomy.csv:rows
scitests	328	measured (re-run, 2026-10-15)	repo-tests.json:suites.science/tests.passed
wltests	42	measured (re-run, 2026-10-15)	repo-tests.json:suites..claude/skills/v2r-loop/scripts/tests.passed
papertests	173	measured (re-run, 2026-10-15)	repo-tests.json:suites.paper/tests.passed
mutlive	48	measured (re-run, 2026-10-15)	repo-tests.json:mutants.live
mutkilled	48	measured (re-run, 2026-10-15)	repo-tests.json:mutants.killed
mutretired	1	measured (re-run, 2026-10-15)	repo-tests.json:mutants.retired

B Reproduction

Each producer refuses to run on a dirty working tree and records whether the tree was clean, so the commit an evidence file names is the code that wrote it rather than merely the commit that happened to be checked out. The science, input, agent-run, sealed-suite and first-pass files were all rebuilt from one such commit:

`26da24de11f4470b7f2ddfb72e72cbae4a382d3d`

Two further script-produced files are measured from later commits — the repository's own test and mutation counts, since a paper cannot be written before the tests it counts, and the plugin suite, re-run against its tag after the check described in Section 6 refused its original record. Every script-produced file names the commit it came from, and the build refuses a claim resting on one that cannot say whether its tree was clean. Two files are not script-produced and so have no producing commit to name: the trace-store count, which an external store answered on the date given, and the taxonomy, which is hand-authored and checked rather than generated. Both are declared as exceptions in the build's own list, with a reason, so neither can pass by being forgotten. At the commit above, the repository's evidence README lists, for each evidence file, the task that produced it and the exact command. Every script records its git commit, timestamp, inputs path, interpreter and library versions beside its output; the model re-run additionally records the model identity, cache revision and weights digest. The sealed suite is re-run with the referee command named

in the README; the referee signs a receipt only for a suite that passed with nothing skipped and every declared result accounted for.

References

[1] CoreWeave. Serverless inference. `https://docs.coreweave.com/products/inference/serverless`, 2026. Documents serverless inference as delivered through W&B Inference and running on CoreWeave GPU infrastructure; accessed 2026-10-15.

[2] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41, 1978.

[3] Sayash Kapoor and Arvind Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9):100804, 2023.

[4] Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, Allan dos Santos Costa, Maryam Fazel-Zarandi, Tom Sercu, Salvatore Candido, and Alexander Rives. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023.

[5] Joshua Meier, Roshan Rao, Robert Verkuil, Jason Liu, Tom Sercu, and Alex Rives. Language models enable zero-shot prediction of the effects of mutations on protein function. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

[6] Siddharth Narayanan, James D. Braza, Ryan-Rhys Griffiths, Manu Ponnapati, Albert Bou, Jon Laurent, Ori Kabeli, Geemi Wellawatte, Sam Cox, Samuel G. Rodriques, and Andrew D. White. Aviary: training language agents on challenging scientific tasks, 2024.

[7] Julie Støy, Emma L. Edghill, Sarah E. Flanagan, Honggang Ye, Veronica P. Paz, Anna Pluzhnikov, Jennifer E. Below, M. Geoffrey Hayes, Nancy J. Cox, Gregory M. Lipkind, Rebecca B. Lipton, Siri Atma W. Greeley, Ann-Marie Patch, Sian Ellard, Donald F. Steiner, Andrew T. Hattersley, Louis H. Philipson, and Graeme I. Bell. Insulin gene mutations as a cause of permanent neonatal diabetes. *Proceedings of the National Academy of Sciences*, 104(38):15040–15044, 2007.

[8] The UniProt Consortium. UniProt: the universal protein knowledgebase in 2023. *Nucleic Acids Research*, 51(D1):D523–D531, 2023.

[9] Jie Wang, Tatsuya Takeuchi, Sadaaki Tanaka, Sang-Kyu Kubo, Tatsunari Kayo, Dongcai Lu, Kouichi Takata, Akio Koizumi, and Tetsuro Izumi. A mutation in the insulin 2 gene induces diabetes with severe pancreatic β-cell dysfunction in the Mody mouse. *Journal of Clinical Investigation*, 103(1):27–37, 1999.

id	what happened	the lab analogue	test?
T01	The gate read pytest exit 1 as 'the test failed' when the test never ran (pytest missing, nothing collected, a syntax error).	A QC step marked FAIL when the instrument never ran — the sample is discarded for a reason that has nothing to do with the	yes
T02	Trace publishing printed a URL and exited 0 while the query store recorded nothing.	An instrument log that says 'saved' while the LIMS holds no record; the run is unrecoverable at analysis time.	yes
T03	The lessons pin was computed exactly and identified nothing: a tree SHA over a store that background hooks rewrite.	A reagent lot number copied from the wrong bottle — exact, logged, and not the lot that was used.	yes
T04	The close commit swept hook-mutated memory files into build-unit commits (git add -A), so a unit's diff no longer described the unit.	A sample contaminated during handling: the assay result is real, but it is not the sample's alone.	yes
T05	`instinct capture` on an existing name discarded the correction and reinforced the stale text, exit 0.	A corrected lab-notebook entry overwritten by the original, with the correction counted as confirmation.	yes
T06	Run records could not say whether a run happened; a seeded fixture in the same schema was indistinguishable from a real record.	A results table with no provenance column — simulated and measured rows look the same.	yes
T07	`register verify-record` returned 1 for UNVERIFIED, and 1 means 'retry the unit' everywhere else in the tool.	Two assays sharing one flag value with opposite meanings; the downstream rule fires on the wrong one.	yes
T08	The hard task closed first try because the implementer prompt named the answer; nothing pins the prompts.	A positive control that answers the experimental question for you.	no
T09	Two build units were not testable in isolation; a park for that reason would have been indistinguishable from one earned by a hard task.	An assay whose negative result cannot separate reagent failure from a true negative.	no
T10	Test isolation depended on pytest's collection order: stubs installed at import time leaked for the whole session (F08).	Benchmark leakage — a held-out set the model has already seen, giving a score that measures the leak.	yes
T11	Five of the guard's own tests could not fail; two of three enforcement halves were deletable with the suite green; the mutation gate credited any non-zero exit as a kill (pass 5).	A held-out metric that stays good after the model is ablated — the evaluation is not measuring the model.	yes
T12	A pre-imported name was a transitive dependency: when the resolution changed, three tests went red (A7b).	An undeclared reagent that arrived in another kit's box; the protocol fails the day the kit changes.	yes
T13	The demo seeder wrote a made-up 23-task, 3-pass fixture into the live run-state directory, over pass 1's own record.	A fixture presented as data — a figure drawn from simulated rows placed in the results directory.	yes
T14	The referee's normaliser recognised two verdicts, so a skipped test read as a pass.	A skipped sample counted as a pass because the plate map had a well for it.	yes
T15	The register and the referee parsed the same test report differently, so one could pass what the other refused.	Two readers of one gel using different thresholds; the band is 'present' or not depending on who looked.	yes
T16	A suite with no declared count: a result the parser did not recognise was dropped, and a dropped result is indistinguishable from a test that never existed.	Missing samples not counted as missing — the denominator shrinks to what was measured.	yes
T17	`referee run --sign` attested that a run happened, not that it passed: a failing suite signed the same receipt as a green one.	A certificate of analysis that certifies only that an analysis was run.	yes
T18	Receipts issued before the fix still attested the property nobody had checked; receipt_version had to be bumped to 2.	Recalling a batch of certificates after the assay behind them is corrected.	yes
T19	`plan_required: true` was rendered 'True' by the config layer, the string compare failed, and the guard turned itself off — for the one consumer who asked for it.	A safety interlock that disengages when set to ON because the switch reports 'ON' in a different case.	yes
T20	A green gate could ship with zero tests executed: an empty test command was 'skip', and the DoD attested around it.	A QC report with the QC step left blank and the batch released anyway.	yes
T21	The referee command's 'could not run' (unknown commit, missing tool, zero tests collected) would have looked like 'the suite failed' — the draft wrapped pytest in `|| true`.	A negative result recorded when the instrument was off — the same channel carries both.	yes
T22	The evidence recorder read the referee's LAST output line and, under --sign, found no JSON: a signed, passing 75/75 run was recorded as NOT MEASURED.	A positive result filed as inconclusive because the parser looked at the footer, not the table.	yes

generated from paper/evidence/taxonomy.csv at d7ea517

Figure 3: The failure-mode taxonomy: what happened, the laboratory analogue, and whether a test now pins it. Drawn from `taxonomy.csv`.